Chapter 12. Safety Engineering for SRE with STPA and CAST

By Garrett Holthaus, Ruben Barroso, and Tim Falzone
with Theo Klein

Traditional incident response focuses on fixing what broke. But what happens when a major outage occurs without a single component failing? This chapter explores how Google is adopting systems thinking and safety engineering to provide a proactive approach to reliability, helping to anticipate and prevent incidents before they occur.

On one eventful day in 2019, if you went to Google Maps to search for places in Japan, you likely couldn't find them. As with other outages, SRE was paged and collaborated with product developers to mitigate the incident. We wrote a postmortem once the issue was resolved and Maps in Japan was back to normal. As postmortems go, this was a very good one. It was thoroughly researched and provided a clear and complete explanation of the underlying problems and the sequence of events that led up to the incident. The postmortem included some discussion of how human decision-making contributed to the incident, without blaming anyone. It identified and prioritized action items, and multiple engineering teams—including SRE—started projects to prevent this kind of problem from occurring again.

While we're never happy when something like this happens, so far this sounds like a successful incident response and learning outcome for the organization. Still, a few people had lingering doubts. Despite the thorough postmortem, the SRE lead and the product development director responsible for the system didn't feel the problems were well enough understood to prevent the same thing happening again. There was something intractable about the extremely complex decision making that led to the Google Maps incident, and they wanted to take a closer look. A particular concern was the way in which the incident occurred without any of the software breaking. There were no failures or errors anywhere in the system leading up to the incident. Beyond some process changes and a general push to develop better mitigations, the postmortem didn't have much to recommend.

This incident exposed the limits of traditional, reactive approaches such as postmortems, production readiness reviews, and unit and integration testing. These approaches are necessary, but they're insufficient to anticipate complex software incidents. It would be easy to simply blame the humans involved for making mistakes, but we know that response is both weak and harmful. The limits of reactive approaches have much to do with *accident causality models*, or *causality models* for short. A causality model is a set of beliefs about why accidents occur. As we'll see later in this chapter, we used a new causality model to gain significant insights about the Google Maps incident in Japan, which led to important improvements in the system.

It's helpful to note that in other industries, the term *accident* means "event resulting in a loss" and *incident* means "undesired state but no loss." By contrast, the software industry frequently uses incident for the former and near miss for the latter. In this chapter, we use the software industry's definition of incident.

Limits of Traditional Causality Models

The causality model that predominates in the software industry is the *linear chain of events*, a model that was first defined almost a hundred years ago. In this model, the cause of incidents is a sequence of component failures (events), with each event resulting in a subsequent event. In Figure 12-1, a hypothetical production service incident is divided into a sequence of events meant to explain why the incident happened. Figure 12-1 also shows measures for preventing each failure event, a strategy commonly known as *defense in depth*.

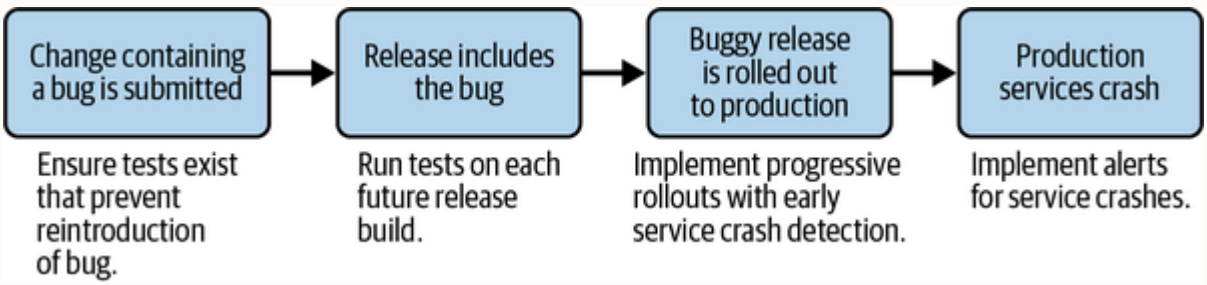


Figure 12-1. A sequence of events in a hypothetical production service incident, along with their preventive measures (defense in depth)

The first event in the causal chain is typically designated the *root cause*. Thus, the root cause of this production service incident is the introduction of a bug in the code. As we'll show later, this assignment of root cause is somewhat arbitrary, and it's almost always possible to continue the causal chain farther back in time—although sometimes this leads to causes that can't easily be classified as events. The focus of postmortems based on a chain of events causality model is on identifying which components behaved "incor-

rectly." It follows that preventing future recurrences of the incident requires implementing measures that "break" this causal chain.

ROOT CAUSES OR CONTRIBUTING FACTORS?

Root cause is a widely used term for describing the factors that ultimately caused an outage. However, parts of the SRE community prefer other terms, like *contributing factors* or *precipitating factors*, because they feel that the term root cause is misleading, especially when used in its singular form.

In complex distributed sociotechnical systems, there is rarely a single root cause for any given problem. Many factors contribute to adverse outcomes. You typically need to find and address all of them to prevent recurrence of the problem. If you stop root causing when you have found the first relevant incident trigger, you will miss other contributing factors. Even worse, in some organizational cultures, the alleged single root cause is found quickly: human error.[1] As SREs, we know that blaming individuals is not going to solve the problem.

After lengthy discussion, we decided to use the term *root cause* and the verb *root causing* in this chapter, despite the outlined issues, to reduce the overhead of introducing new terminology. We don't believe that renaming will solve the actual organizational culture problem. We stress that there are almost always multiple contributing factors to an incident, just as a weed often has a complex root system. However, as we'll show in this chapter, we can gain even deeper insights into system design flaws by adopting a new set of beliefs about why incidents occur.

When the assigned root cause for a chain of events involves a human's actions (or lack of action), root cause analysis can be tricky. Even in a blameless culture, avoiding individual blame should not hinder us from understanding the reasons for individual actions. Our learning is not complete if we stop at "change containing a bug is submitted." The effectiveness of our corrective actions is the true measure of our learning. Therefore, we should ask a more precise question: will a simple measure, like ensuring a test catches a specific bug, prevent similar future incidents?

The primary limitation of the chain of events model is its flawed assumption that a sequential order of events implies a causal relationship. In complex systems, eliminating a preceding event is not sufficient to prevent the subsequent event, as there may be other ways in which that event can happen. Additionally, while a timeline describes what hap-

pened, it fails to explain the *causal factors* behind the events. Causality in complex incidents is more nuanced than simply identifying which event came before another.

For example: why did the engineer submit code that contained a bug? It could be due to a combination of factors, such as:

- The engineer's coding skills
- The engineer's mental model (i.e., their perception of how the system works, including the release and rollout process)
- The software development practices used in the company (e.g., testing, code reviews)
- The software development tools used in the company
- The complexity of the code
- Environmental pressures, such as launch deadlines or workplace stress

Analyses based on chain of events causality rarely ask these questions, because their primary purpose is simply to identify the root cause or causes. This reflects the human impulse to find simple explanations for complex incidents, sometimes referred to as *root cause seduction*.[2] The flaw in this way of thinking is that causal factors often cannot be arranged into a linear chain.

We will now explore an alternative model for understanding why accidents or incidents occur.

STAMP: A Modern Causality Model for Complex Systems

As we've established, the chain of events causality model is insufficient to analyze today's highly complex systems—particularly software, which can consist of billions of lines of code. We need a modern causality model—and we have an effective one based on systems theory.

The field of systems theory gained traction starting in the 1950s, when people realized that it was impossible to fully analyze the properties of increasingly complex systems by breaking them down into component parts. System properties that emerge from the interaction of many components, such as services, products, infrastructure, and the humans who interact with them in a software company, are often referred to as *emergent properties*. These properties can only be understood by analyzing the system as a whole, not the individual components. Importantly, *safety is an emergent property*. For example,

given the specification for an API, it's impossible to tell whether it is safe without understanding the system around the API and how the API will be used in that system. In other words, to truly anticipate the incidents that might occur with the API, you need to look at it in the context of the whole system.

System-Theoretic Accident Model and Processes (STAMP), is a modern causality model that attributes incidents not to a chain of events but to a system design that doesn't safely constrain its components' behaviors. In STAMP, an event results from multiple causal factors influencing a component's behavior. A *causal factor* is an element within the system's design that can lead to unsafe behavior—either by humans or by automation—and possibly to an incident. STAMP fundamentally rejects the notion of a single root cause. Instead, it recognizes that incidents are rarely caused by a single human mistake or a single component failure. More often, they are caused by system design flaws that allow unsafe behaviors to emerge. Human and software errors are therefore symptoms, not causes, and preventing these requires a focus on the larger system in which people and software operate.

Figure 12-2 illustrates this with the hypothetical production service incident we introduced previously.

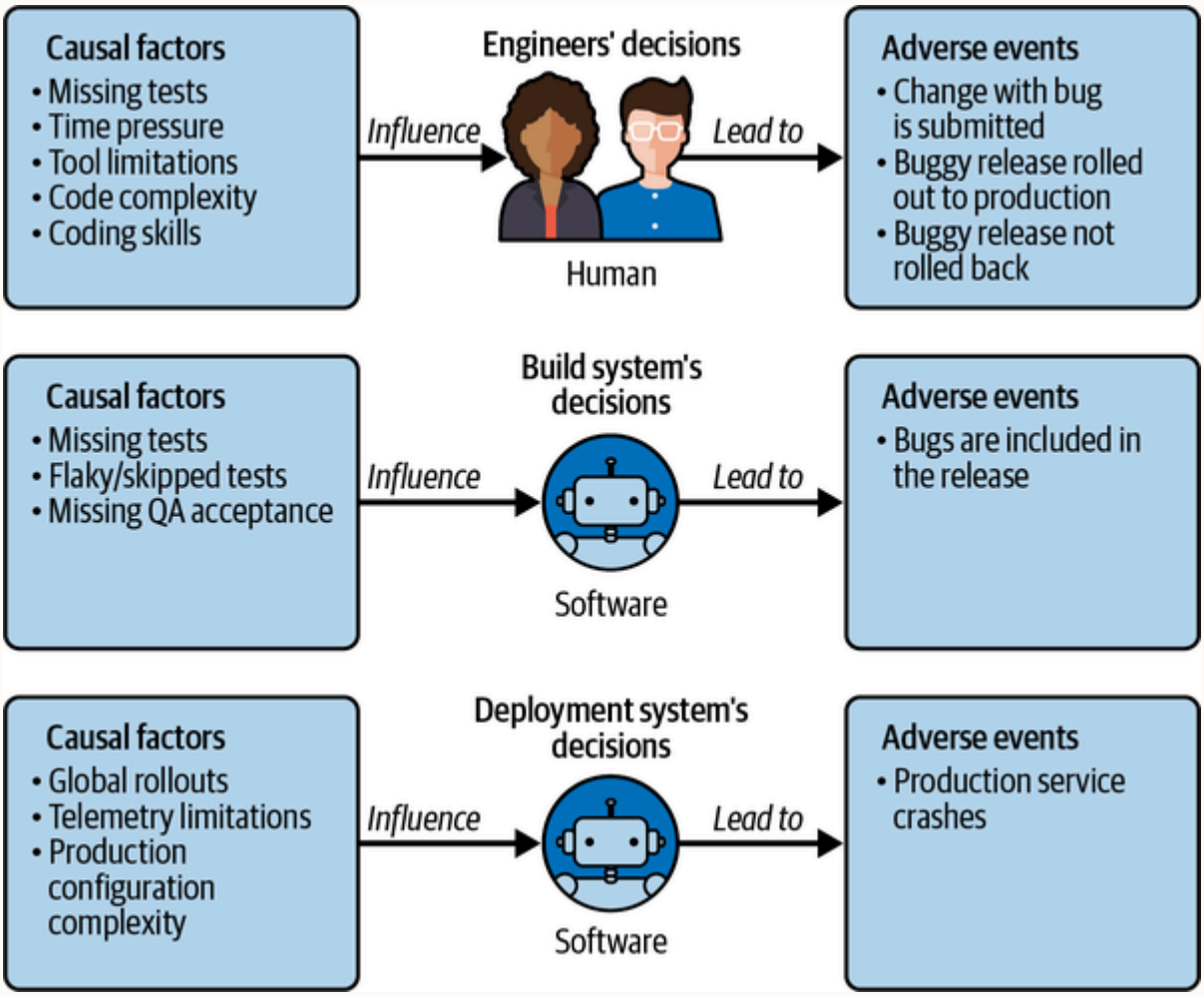


Figure 12-2. Using STAMP to identify causal factors affecting component behaviors that lead to adverse events

Any event can be assigned a timestamp, allowing for chronological ordering. However, it's important to note that this ordering does not imply a causal relationship between events, unlike a linear chain of events.

What are some possible causal factors that could lead to rolling out a buggy release in the hypothetical production service incident? Under the STAMP model, the engineers' decision was not a causal factor. Rather, the system design enabled causal factors to influence their decision. Examples of causal factors might include:

- Missing tests prevented the engineers from discovering the bug.
- The bug went unnoticed because the reviewers couldn't effectively assess the service's behavioral changes due to the complexity of the codebase.
- Global rollouts prevented the engineers from learning about the bug's effects until it was too late.
- Management pushed for an aggressive launch schedule for a new feature.

These causal factors explain why the decision to proceed with the rollout, while in hindsight unsafe, appeared reasonable to the engineers at the time.

STPA and CAST: Analysis Methods Based on STAMP

Now that we have a causality model that's better suited for today's complex software-based systems, we need methods for analyzing system safety based on this model. There are two approaches to take here. *Hazard analysis* is forward-looking, focusing on trying to figure out all the ways an accident or incident can happen for a given system design. *Accident analysis* is backward-looking, aiming to figure out why an accident or incident occurred.

STPA: Forward-Looking Hazard Analysis

The forward-looking method based on STAMP is System-Theoretic Process Analysis (STPA). STPA is a top-down hazard analysis method that identifies systemic flaws, not just component failures. As defined by STAMP, the core idea is that accidents happen due to inadequate control over the way interactions occur in a complex system. Consider the hypothetical production service incident: there were many interactions between humans and software, and then internally between large software systems, that enabled the incident to occur. At each of those interaction points, the system design could have applied a constraint that prevented further damage. The key idea in STPA is that you can

carefully study those interactions to identify and apply the proper constraints before anything bad happens.

As a proactive method for assessing system safety, STPA is unique in that you can run it when all you have is a design on paper. A robust system design prevents an unacceptable outcome even if the engineers try to submit code containing a bug.

CAST: Backward-Looking Accident Analysis

The backward-looking method based on STAMP is Causal Analysis based on Systems Theory (CAST). CAST refocuses the goal of accident analysis (or postmortems) from a search for failures to identifying why the system as a whole didn't prevent the incident from occurring. CAST differs from other postmortem methods in three important ways:

- It aims to understand why system design enabled unsafe behavior, seeing causality as a complex process arising from interactions among components, not a linear chain.
- It explains incidents by focusing on why decisions made sense at the time, rather than blaming individuals. CAST aligns with a blameless postmortem culture.
- It focuses on system design, viewing human error as a symptom of flawed design, not the root cause of an incident.

CAST enables a deep understanding of causal factors, which allows us to redesign systems to change or eliminate the behavior that led to the incident.

Table 12-1 summarizes the differences between the STPA and CAST methods. The following sections dive deeper into each one and walk through examples of their use.

Table 12-1. Comparison between STPA and CAST

	STPA	CAST
Focus of the analysis	Hazard analysis method	Accident analysis method
Forward- or backward-looking	Forward-looking: anticipate incidents before they occur	Backward-looking: analyze the causes of an incident that occurred
Who is involved	During the analysis, work with system owners (subject matter experts)	During the analysis, work with people involved in the incident, including owners of systems involved
Scope	Existing production systems and proposed system designs	System as it was during the incident

STPA: A Method for Anticipating Incidents

Here, we present a high-level overview of the four steps of STPA. We'll use the hypothetical production service incident to illustrate some parts of the process.

Step 1: Scope the Analysis

In step 1, we scope the analysis and define the unacceptable outcomes the system must avoid. Scoping the analysis involves identifying the stakeholders who are invested in the system, including SREs, developers, the company, customers, users, and so on. These stakeholders then define the overall goals of the system, and the *losses*, or unacceptable outcomes, they want to avoid. For many software products, these are high-level business outcomes such as loss of the ability to meet the goal of the system—we typically call this a *loss of mission*. An example loss of mission would be if a Google Maps user loses the ability to navigate in the real world. Other losses include loss of revenue, loss of brand trust, etc.

The next part of this step is the identification of *hazard states*. A hazard state is a detectable system state that could lead to losses under worst-case conditions. For Google Maps, one hazard state is when the published map is inconsistent with the real world. This state doesn't necessarily lead to a loss—for example, if we catch it quickly and correct it before it impacts users—but as time goes on, the chances of negative effects increase. (For more on hazard states and why they are so useful for SREs, see Tim Falzone and Ben Treynor Sloss's *;login:* article "The Evolution of SRE at Google".)

Step 2: Model the System

In step 2, we build a *hierarchical control structure*, or *control structure* for short, which models *functional* behavior in a system. This is a key difference from other kinds of models that are common in software engineering contexts. Typically, the types of models SREs create and work with are *structural models* of the system—they describe class hierarchies or data flows between network-connected services. In contrast, an abstract functional model captures behavior, grouping different structural elements that serve a common behavioral purpose. There might be five different teams that can all deploy updates to a running server. Functionally, we can group them together into a single abstracted entity that deploys updates, even though they may be organized very differently, use different mechanisms, and receive different feedback.

STPA calls these abstracted functional entities *controllers*. Each controller has responsibilities intended to prevent some of the hazard states. For example, there must be a controller in Google Maps with the responsibility of ensuring the published map is consistent with the real world. To decide what actions to take, software and hardware controllers have a *control algorithm*—e.g., how to update the published model—and humans perform *decision making*. Controllers also have a current belief about the state of the system, called a *process model* (or, for humans, a *mental model*)—e.g., believing that the published model is currently consistent with the real world. Note that this belief might at times be incorrect!

Each controller takes actions—referred to as *control actions* in STPA—to meet its responsibilities. These actions are taken based on the control algorithm and mental or process model (correct or not), which is updated by *feedback*. The safety of a decision comes down to the correctness of the control algorithm or the decision making and the accuracy of the mental or process model as compared to the true system state. This accuracy depends on the feedback that updates that model (i.e., whether it is correct, complete, and timely).

MANAGING COMPLEXITY WITH HIERARCHICAL CONTROL STRUCTURES

Typical software systems are so complex that it is almost impossible for humans to understand them deeply enough to reason about possible design issues. Hierarchical control structures, through their use of abstraction and control–feedback loops, enable complex systems to be represented by a small number of controllers. We typically see Google analyses with 5–10 boxes—this is in contrast to a dataflow diagram, which might have 10 times as many boxes and arrows, and doesn't typically include the humans in the system. We find that the act of bringing together SREs and product developers from several different teams to build a control structure frequently results in a deeper shared understanding of the system among all involved. The control structure then becomes a common language when considering further changes or feature additions, reducing the possibility of design issues due to separate teams understanding the system differently.

Using the hypothetical production service incident as an example, let's look at the fundamental building block of a control structure: a single control–feedback loop (shown in Figure 12-3). Specifically, we'll examine the decision of whether to roll back the production service to the previous software version, after the release version containing the bug was rolled out.

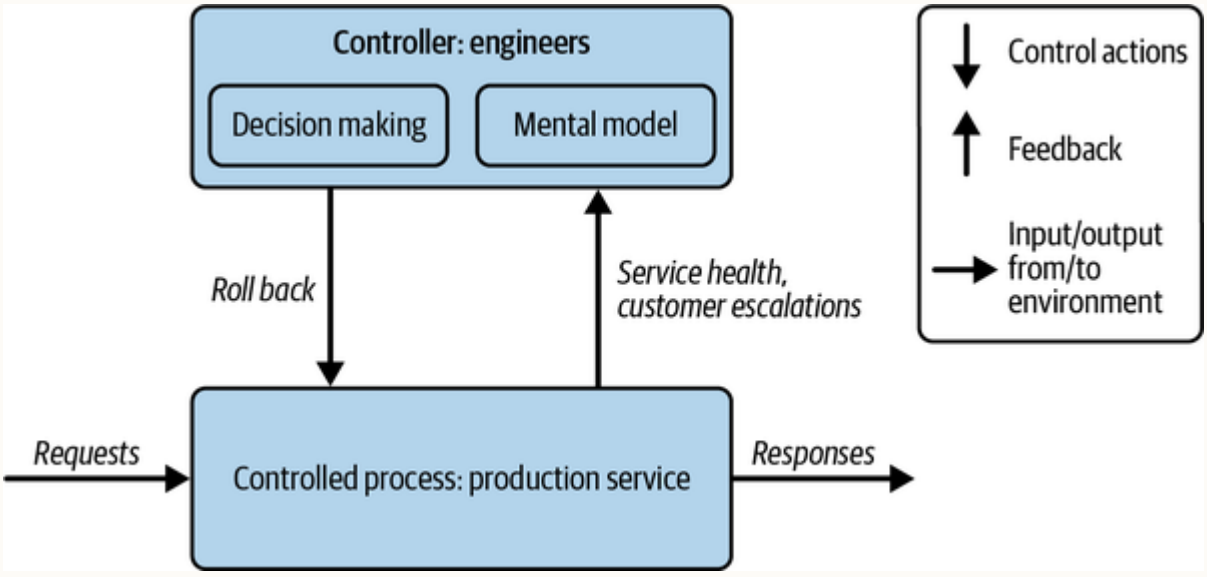


Figure 12-3. The basic control–feedback loop used to model systems in STPA

This simple loop, including a human decision maker (the "engineers" controller) that controls a part of the system (the "production service" controlled process), is all you need to analyze. A typical control structure is made up of multiple such control–feedback loops. In this simple example, the engineers (who could be a product development team, SREs, or both) follow a decision-making process about when to roll back to a previous version. For example:

IF I see a combination of alerts and customer escalations indicating production service unhealthiness, followed by a quick correlation between the issue's onset and a recent version rollout

THEN I roll back the recent version because I believe it will mitigate the incident.

Later, in step 3, we will thoroughly analyze the contexts in which this decision to roll back can lead to the system entering the hazard states defined in step 1. But just from this simple model, we can see that the safety of the decision depends on what the engineers believe the state of the production service to be—in other words, their mental model. This mental model is informed by feedback coming from production itself, including error responses and customer escalations. The choice of whether to roll back is also guided by whatever they rely on in the decision-making process (e.g., procedures, playbooks, expert knowledge), and it is directly influenced by their responsibility to maintain service performance goals (availability, latency, etc.). A rollback is often the required response when these goals are compromised by the current state of the production service.

Let's say our production service implements an API. The primary goal of this system can be summarized as "API request fulfillment." This requires the production service to meet certain criteria: responding to all incoming client requests within the specified latency limits, ensuring all responses conform to the API specification, maintaining a low rate of server-side error codes, ensuring the correctness and freshness of data in the response, and so on.

Without an engineer *controlling* the production service, the goals of the system could be violated for any number of reasons—a software bug, an environmental condition (e.g., a networking or hardware disruption), a pathological request (e.g., an overly large request, an unauthorized request), etc. We can think of this as the controller *constraining* the behavior of the controlled process to ensure the system meets its goals. As well as goals of its own that it seeks to achieve or maintain, the controller has a responsibility in the system, and it takes actions to constrain the controlled process in order to fulfill that responsibility. In this example, the engineers have a responsibility to ensure the production service returns error-free responses to incoming requests. To fulfill this responsibility, the engineers constrain the behavior of the production service to keep it from entering an unsafe state. One way they do that is by rolling back the erroring production service to a previous version.

Even without taking the STPA analysis further, this model contains rich information that is missing from a structural network model. The dynamic feedback loop helps us think

about *why* a rollback might occur. We can compare this abstract functional model to the real system and ask questions like:

- Who in the system has the responsibility to roll back? Is it more than one person or team?
- Do each of those entities use the same decision-making process?
- Do they take into account the same criteria to decide to roll back?
- Do they receive the same feedback?

Step 3: Identify Unsafe Control Actions

Step 3 uses this kind of questioning and formally explores all the possible ways the decision making can go wrong. We analyze each control action (decision) in the model, one by one. Context is critical—control actions can be safe or unsafe depending on their context. STPA focuses only on the contexts in which the control action can lead to one or more of the hazard states identified in step 1. It calls these *unsafe control actions* (UCAs).

By itself, the decision to roll back or not roll back the production service to a previous version is neither safe nor unsafe. However, not rolling back the production service to a previous version *when the current version is causing errors in excess of acceptable levels* (the all-important context) causes a hazard and violates the API request fulfillment goal of the system. The timing of a control action should also be considered. Depending on the context, performing an action too early or too late (or for the wrong duration) could also be unsafe.

By studying all the control actions in the system, we can anticipate other UCAs that can contribute to an incident.

Step 4: Build Causal Scenarios

Finally, in step 4, we come up with *causal scenarios*. Why might a software controller or human in the system take an unsafe control action identified in step 3? Did they get bad feedback, or was the feedback they needed to make a safe decision missing completely? Did they have an inaccurate process or mental model about the current state of the system? If so, why? Did they try to do the right thing, but their safe action was overridden or lost due to a breakdown in communication? Or maybe they did the right thing, but the system reacted in an unexpected, unsafe way.

For the hypothetical production service incident, we identified an example UCA:

Engineer does not roll back service to previous version when the service is serving errors in excess of acceptable levels.

The engineer might or might not know about the errors, but for this example, let's assume they do. So, the general scenario is that the engineer does not roll back the service in spite of knowing there are errors. Now, we can present this scenario to the system experts and brainstorm why this might happen. Here are several possible reasons (but this list is not exhaustive):

- The engineer defaults to debugging the issue, which takes an indeterminate amount of time, instead of reflexively performing the rollback and debugging later.
- The engineer is aware of changes in the system that make the previous version incompatible. Their only option is to roll forward a fix, which takes time.
- The engineer does not have the necessary permissions to roll back the production service to the previous version.
- The release system was launched recently, and the engineer is not familiar with the new way to roll back production services.

These causal scenarios anticipate exactly how an incident can occur. Armed with a list of these scenarios, we can now prioritize them, engineer solutions that efficiently cover the scenario space to prevent broad categories of incidents, and make thoughtful decisions about which scenarios we can simply detect and resolve manually. The ultimate goal is to ensure the system doesn't incur one of the losses we listed in step 1.

Next, we'll walk through a real-world example of how this model can be applied to incident analysis.

Applying STPA to the Maps Data Updates Approval Process

Following a major incident, retrospective CAST is the typical analysis choice in fast-paced tech environments due to its speed and lower cost. Factors influencing this include:

Limited participation

CAST requires only subject matter experts (SMEs) with direct knowledge of the incident or the involved systems, minimizing the number of participants. STPA, con-

versely, often requires broader representation across different teams and roles, raising the bar for leadership approval.

Restricted scope

CAST focuses narrowly on the specific decisions that contributed to the incident (depth analysis). STPA is a depth and breadth analysis of all decisions that could cause hazards.

However, for the incident we discuss here, the team chose to run STPA. They wanted to see if STPA could find the same flaws that the postmortem did—and, importantly, whether they could gain these insights from a forward-looking method that could be applied proactively, before an incident happens. In addition, the team wanted to more deeply understand the hazards involved in the incident, which they thought might help them identify other hazards and causal scenarios and potentially enable them to prevent future incidents. They engaged Dr. John Thomas an STPA expert and the executive director of MIT's Partnership for Systems Approaches to Safety and Security, to guide their effort. They then spent a few weeks learning and executing STPA.

The Google Maps Incident in Japan

Before giving an overview of the Maps data updates approval STPA, let's review what happened in the incident.

In 2019, Google embarked on an effort to shift from using third-party data to using Google intellectual property for the map of Japan. On the surface, the Maps incident appeared to have been caused when Google made a large update to map features representing Japan's internal administrative divisions (e.g., prefectures, municipal divisions). These geographical features are a nested set of polygons containing all the other features in Japan. That is, every other feature on the map—schools, roads, parks, office buildings, etc.—has relationships to these administrative features. We use these relationships during searches to ensure we return the correct result. For example, a map feature might be in Chūō Ward, which is in Osaka Prefecture, which is in the Kansai Region of Japan. The incident occurred when all of the connections between places and their administrative divisions were dropped. Without these relationships, search results were confusing at best and totally wrong in the worst cases.

The data process that updates Maps data is dynamic. After features are updated, a daily repair pipeline takes a snapshot of the current state of Maps, verifying and, if necessary, adding, removing, or updating connections between features. In the incident, the humans updating Japan's administrative divisions *intentionally* dropped all existing connec-

tions between geographical features (such as schools, roads, etc.) and their associated administrative features. The rationale was sound: given the scale of the update, it seemed safer to clear all connections and rely on the repair pipeline to re-create them rather than to attempt to re-create them manually. The team knew some features would temporarily lack administrative associations, meaning they would no longer appear in searches. However, they assumed the daily repair pipeline would execute immediately and reinstate these connections. From the team's perspective, the updated features looked correct. The incident occurred because:

- Due to dependencies in the data, the update was broken down into smaller sets of changes, which were serialized and submitted independently. This caused dashboards to show a gradual decline in consistency, which was insufficient to bring attention to the problem.
- The first set of updates was applied right after the daily repair process had finished. This meant the dropped connections would not be repaired until the following day, leaving search functionality impaired.

Applying STPA

Let's now examine how the team used STPA to more deeply examine the Maps data updates approval process and discover additional causal scenarios. The following account is from the perspective of the STPA analysis team.

Step 1

In step 1, we identified the following hazard:

> *The published map is inconsistent with the real world.*

When the batch of updates that dropped all administrative connections was applied, the Maps data updates approval process entered this hazard state. Again, this hazard will lead to a loss only under certain conditions—in this case, if users try to search for features in Japan that don't have connections to their administrative divisions. We can design our system to prevent this hazard state, or, if this isn't practical, we can at least ensure that we can detect the hazard and quickly mitigate it.

SUCCESS STORY: MEASURING INCONSISTENCY

To understand this success story, it is important to know that Google Maps uses an eventually consistent data model to build a "digital twin" of the real world. This data model is periodically (and frequently) published to Google's search index. The design-time assumption was that any inconsistency in the data model—and therefore the hazard—would be localized in both time and space, with processes constantly running to repair inconsistencies. While interviewing the system experts, the STPA analysis team realized that no one had ever tried to validate this design assumption. Among the many insights the STPA process produced, one of the biggest surprises was that both parts of it turned out to be wrong: inconsistencies affected very large numbers of features, sometimes for periods lasting multiple weeks. One of the recommendations that emerged from the process was to build a simple heuristic to measure inconsistency levels in real time.

Step 2

In step 2, we created a control structure for the system. We modeled the key software and human control–feedback loops. As with many complex systems, humans were making important decisions at the perimeter of the system design. Figure 12-4 shows a section of the control structure that models the human Maps data updates approval process.

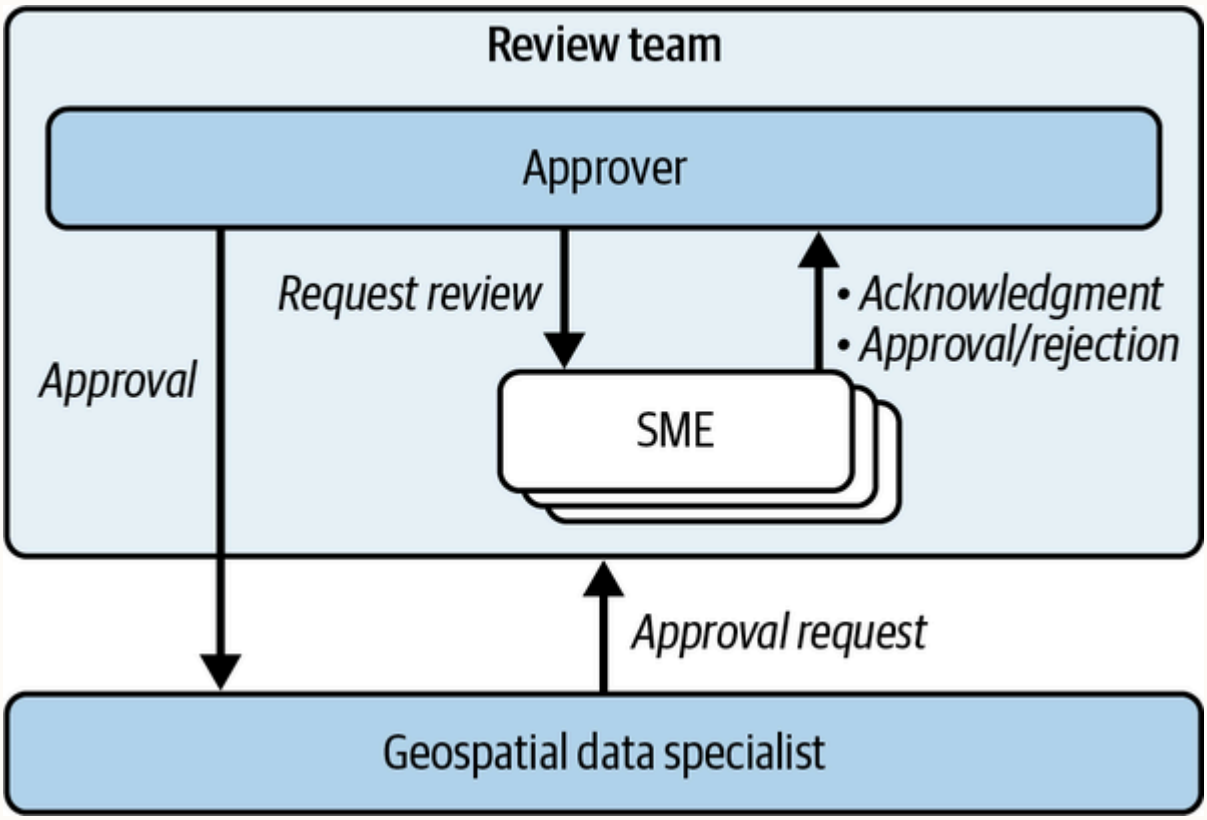


Figure 12-4. Control structure for the human Maps data updates approval process

Complex Maps data updates go through a review and approval process involving several teams of people. First, a geospatial data specialist prepares the change. Once the change is ready, they submit it to a team of reviewers. This team consists of two groups: approvers and SMEs. The approvers are ultimately responsible for the decision to publish the change. They often work with local SMEs in the geographical areas included in the update, to ensure the changes are correct.

Human geospatial data specialists handle particularly tricky, often very large-scale changes that need to be carefully prepared and evaluated. As we mentioned earlier, one such change was updating all of the administrative divisions in Japan. The geospatial data specialists performed many rounds of data evaluation to verify that the update would result in a correct map in Japan. Once they believed the change was ready to submit, they followed the normal procedure of getting the data update approved for publication by the team of reviewers. Those reviewers in turn reached out to SMEs based in Japan to help them verify the correctness of the changes. This crucial set of interactions between the geospatial data specialists, the approvers, and the SMEs had never been modeled before and was not analyzed as part of the postmortem following the incident. However, we did analyze these interactions with STPA.

Step 3

To analyze this small part of the control structure, we examined each of the control actions (the downward arrows in the model). For each control action, we wrote down contexts in which the action would lead to a hazard state, coming up with a list of unsafe control actions.

One example of a UCA we generated for the "Approval" action was:

> *The review team approver provides approval when the data update is not approved by the SMEs.*

It's unsafe for the review team to approve changes for publication if the SMEs haven't approved the changes. This UCA could lead to the hazard state of Maps being inconsistent with the real world.

When we wrote this UCA, it didn't seem very plausible. Why would the review team ever do this? However, when we asked this question in step 4, we got some interesting responses.

Step 4

In step 4, we went through each of the UCAs, building specific causal scenarios that explained how the UCAs might occur. The annotated control structure in Figure 12-5 shows how STPA enables us to methodically generate questions that often lead to the discovery of system gaps or vulnerabilities. In this case, we can ask questions about the approval process (e.g., What does the reviewer need to know before granting approval?) and questions about the feedback from the SMEs (e.g., What if the feedback is late or incomplete?). Figure 12-5 shows how the control structure helps us organize and ask questions about various aspects of the dynamic process for approving updates.

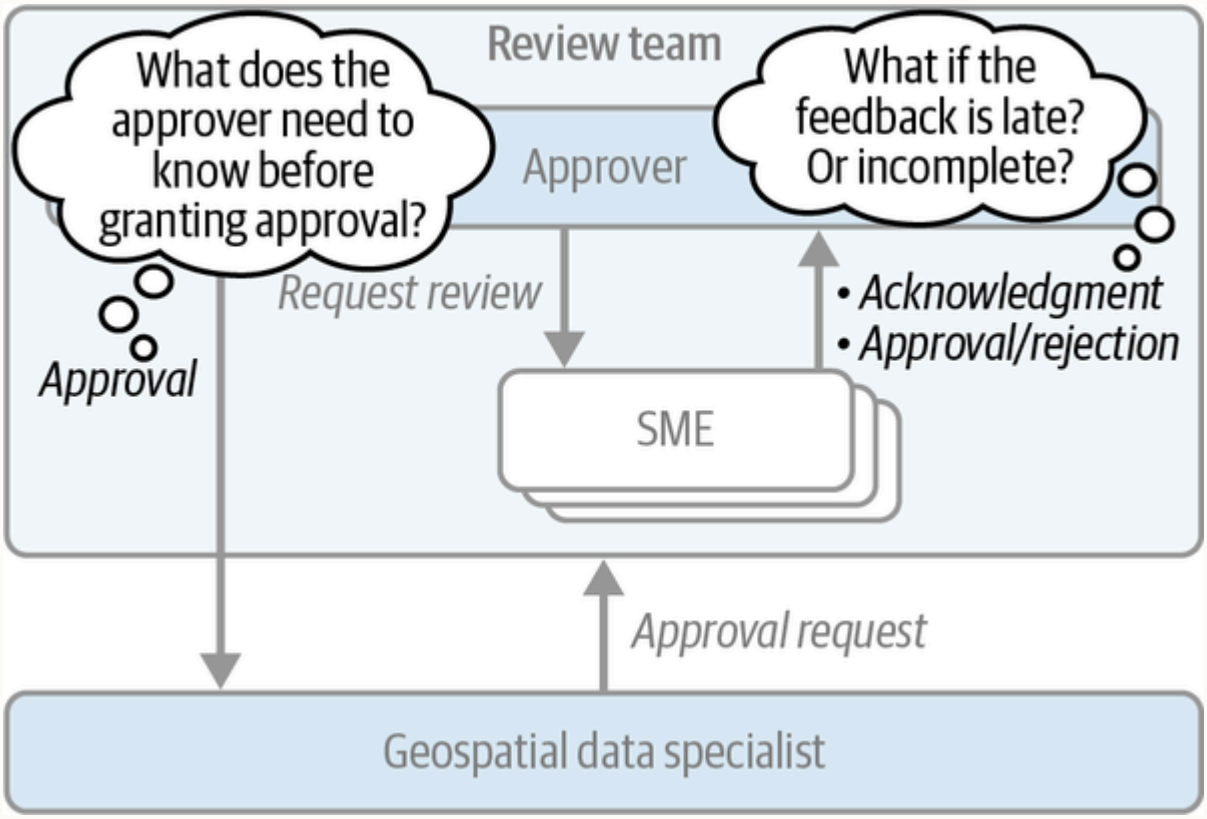


Figure 12-5. Using the control structure to generate questions about the Maps data updates approval process

In developing these causal scenarios, we met with a member of the review team to ask about the approval process. We asked whether there were any circumstances in which the review team would grant approval to publish a data update without an explicit approval from the SME. The answer was very surprising. The review team had an unwritten but established procedure: they aimed to gather feedback on changes within 24 hours. If no concerns were raised during this period, the change was approved. They explained that this procedure had developed over time in response to pressure to improve the freshness of data on the map.

As we kept digging, we learned that the review team was also operating with some important assumptions. One assumption, mentioned in step 1, was that any issues that did come up could be quickly identified and corrected. Another was that any new data added

to the map probably resulted in a net improvement in data quality. Each of these assumptions was incorrect to some degree, as they were not in sync with the capabilities of the system. The product development teams also held incorrect assumptions about the degree of validation and vetting done on big data updates; these led them to falsely believe that the set of defensive checks at the time was sufficient to handle any problematic data updates.

These findings were a surprise to us, but they helped explain what had happened during the Maps incident. Applying STPA gave us a more detailed understanding of the causal factors. The geospatial data specialist prepared and evaluated the large update to Japan's features, which they later sent to the review team for approval. The review team in turn sent a request to a subject matter expert in Japan for help in reviewing the update. However, the US-based reviewer sent the request on a Thursday, and the request arrived in Japan on Friday, which happened to be a holiday. The SME didn't see the request until the next week. When they reviewed the changes on Monday morning, they immediately replied that the update should not be submitted. Their experience indicated that the automated repair of missing connections might take a few days. This directly contradicted the initial assumption made by the review team and the geospatial data specialist, which was that the connections would be fixed quickly after the data updates were applied.

However, back in the United States the 24 hours had elapsed, and the review team had already approved the update. By the time the feedback from the expert in Japan reached the review team, the geospatial data specialists had already started submitting the changes, and the system had been in the hazard state for some time.

This was clearly the wrong outcome, and in hindsight it's easy to see how this decision making was flawed. However, no one had ever documented any of this. These questions had never been asked until we decided to run STPA. Although clearly we need to make the software as robust and resilient as possible, we are working at a significant disadvantage if the human decision-making process is unsafe. Starting with the review process, we were able to determine several ways to strengthen it. Adding these to all the other system improvements STPA led us to make (e.g., implementing monitoring and alerting for data inconsistency) resulted in an overall system design that was much safer than before the analysis.

Results and return on investment

The entire STPA process was carried out by two engineers, who worked on the analysis part-time for five weeks with support from an expert. We analyzed more than 30 major

software subsystems and studied the interactions among over a dozen different teams of engineers. STPA identified 56 causal scenarios that demonstrated gaps in the system. We compared those gaps with past events, especially the Maps incident in Japan that had occurred several months earlier. STPA identified almost all of the defects found in the postmortem. The additional defects it uncovered could have led to at least four other major incidents (and many other moderate or minor incidents).

Because STPA identified so many scenarios, it produced many more recommendations to improve the Maps data updates approval process—34, compared to only 7 from the postmortem. This enabled the product developers to design effective solutions that covered a much larger portion of the scenario space. By contrast, spending a similar amount of effort addressing the relatively small number of defects identified in the postmortem could have left many of the STPA scenarios unmitigated.

For a relatively small investment, the rate of return was terrific. This was also one of the *slowest* STPA projects we've ever conducted, because we had to learn the method as we went. Since then, we've seen even better rates of return as we have gained experience with STPA and streamlined the process. More importantly, we've been able to proactively identify and remediate design flaws in complex software systems before they led to incidents.

CAST: A New Method for Postmortems

Postmortems have long been a core part of Google's SRE culture.[3] Any software company that runs postmortems has ample opportunities to learn from incidents, and in systems involving both technology and humans, the causes leading to an incident can be very complex. Even with the significant time and effort that many companies spend on postmortems, incidents continue to happen. This can create a feeling that we are constantly addressing individual symptoms without addressing the underlying causes.

This situation doesn't stem from technical incompetence or an unwillingness to dig deep. Rather, it results from using an inadequate accident causality model, such as the chain of events model. If we believe that incidents happen because one event leads to another in a linear chain, our analysis will focus solely on identifying why the chain was not stopped, and we'll ignore other causes that can't be represented as events. An inadequate accident causality model limits our analysis, which in turn hinders our learning. As a consequence, we may confidently generate improvement plans that treat only symptoms, not the actual causes of the incident.

As we discussed earlier, the STAMP causality model is better suited to highly complex systems such as modern software ecosystems. CAST is a retrospective accident analysis (postmortem) method based on STAMP. It refocuses the goal of postmortem analysis from a search for failures to an exploration of why the system as a whole did not prevent the incident from occurring.

In this section, we describe our application of CAST to a different Google Maps incident.

The Case of the Misnamed Cities

One day in 2024, we began receiving user reports about US city names being displayed incorrectly in Google Maps, with some additional text appended. For example, instead of simply "Louisville" or "Indianapolis," users were seeing "Louisville/Jefferson County metro government (balance)" and "Indianapolis city (balance)."

SRE responded to the incident, and after everything was resolved, the engineers conducted a postmortem. As discussed in the previous case study, Google Maps maintains accuracy by having internal geospatial data specialists evaluate all incoming data, from both internal and external sources, before it's imported. The postmortem found that the "(balance)" tags came from a US Census Bureau dataset that we imported into Google Maps. Figure 12-6 graphically represents the traditional chain of events model used in this postmortem, illustrating the events both leading up to and following the incident.

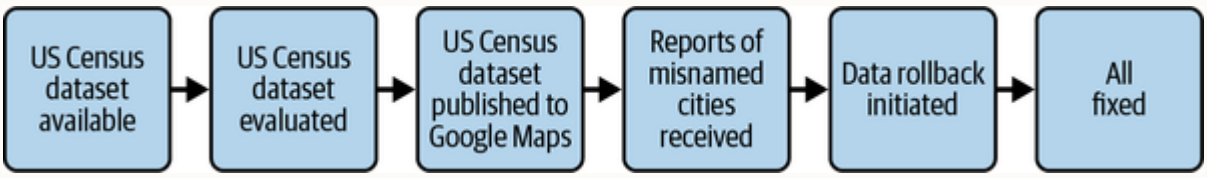


Figure 12-6. The order of events for the misnamed cities incident

The postmortem described the individual events in detail:

- The Census Bureau published a new dataset containing US city names.
- The geospatial data specialists evaluated and approved the dataset for publication.
- Users began reporting the misnamed cities, triggering our incident response.
- We initiated a data rollback of the entire dataset.
- All misnamed cities were eventually fixed.

It identified two root causes, the first being imperfect data sampling. Because a full manual review was not feasible due to the immense volume of data, the geospatial data specialists limited their review to four states where they had local expertise (WA, CA, TX,

and NY). This sample did not include any of the six states with misnamed cities (MT, KS, IN, KY, TN, and GA), as shown in Figure 12-7.

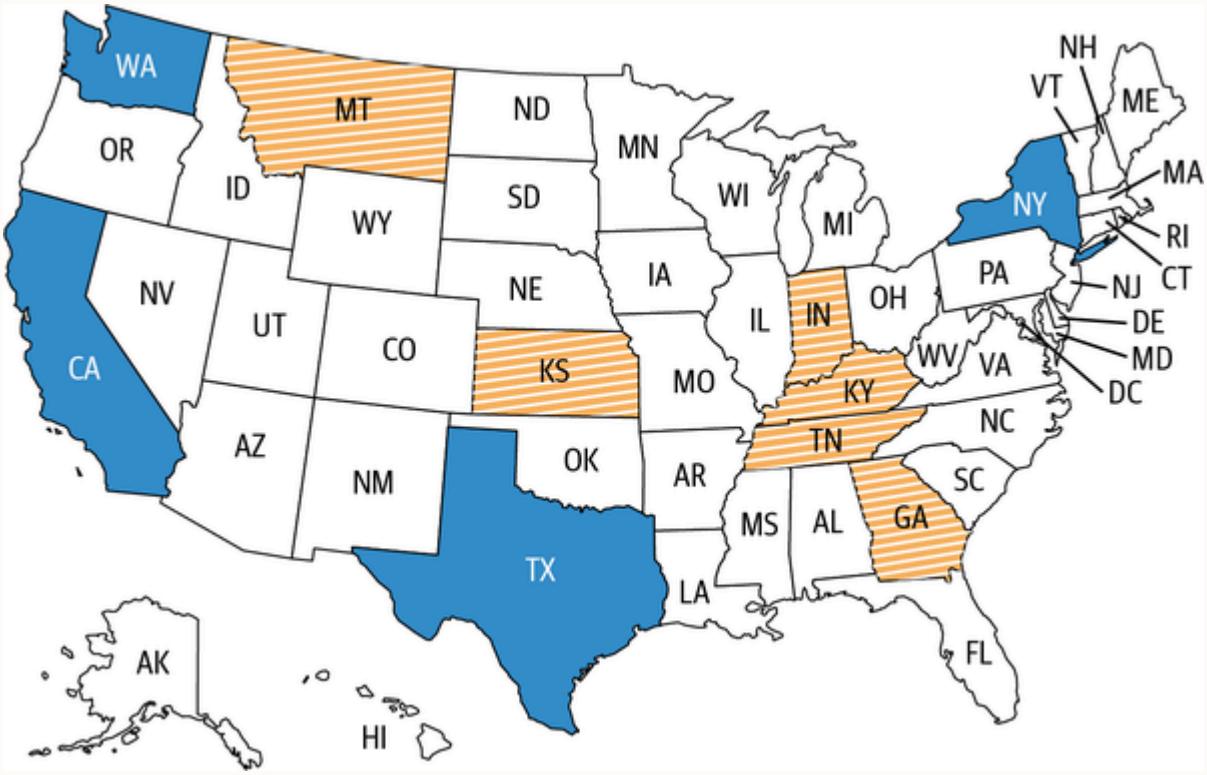


Figure 12-7. Of the 50 US states, 6 contained misnamed cities (striped shading). The 4 states sampled for evaluation (solid shading) did not overlap with any of the states that contained misnamed cities. [Adapted from an image generated at mapchart.net and licensed under CC-BY-SA 4.0]

The second root cause the postmortem identified was inadequate evaluation tooling. The internal tools used by the geospatial data specialists to detect problems in datasets were not designed to detect incorrect city names.

The postmortem produced the following recommendations:

- Automate the detection of incorrect city names within the evaluation tooling.
- Implement programmatic checks to prevent storing invalid city names in the database.
- Write clear instructions for enabling specific checks in the tooling, such as flagging special characters like `/` or `(`.

The recommendations from the postmortem directly addressed the identified root causes. However, as with the STPA example described in "Applying STPA to the Maps Data Updates Approval Process", the team hypothesized that further analysis would identify additional causal factors and opportunities for design improvement not found in the postmortem. In this case, they decided to do a CAST analysis.

Applying CAST

This section analyzes the misnamed cities incident as a case study to demonstrate how each step in CAST is applied.

Step 1: Assemble basic information

The first step of CAST involves gathering basic information about the incident to establish the scope of the analysis. As with STPA, there are two key outputs:

Losses

Unacceptable outcomes realized during the incident

Hazards

System states that can lead to losses under worst-case conditions

During this step, an initial chronological timeline of incident events is compiled and then used as the basis for asking general questions about the events. We avoid assigning responsibility.

Similar to the postmortem, the CAST analysis produced a timeline of key incident events and several initial questions, such as:

- Why evaluate only four states, and how were they chosen?
- Why was this problem not detected internally?
- How confident were the responders that all incorrect city names were fixed?

More detailed questions were developed in later stages.

In this incident, the hazard was the same as for the previous STPA example:

The published map is inconsistent with the real world.

As we'll see, however, the causal factors for this incident were different.

Step 2: Model the control structure

As we've seen, STAMP views complex incidents as a result of an ineffective control structure. In CAST, we model the system's control structure at the time of the incident, focusing on the responsibilities of the controllers. Confusion or inconsistencies in these responsibilities often point to critical causal factors.

The control structure at the time of the misnamed cities incident is shown in Figure 12-8.

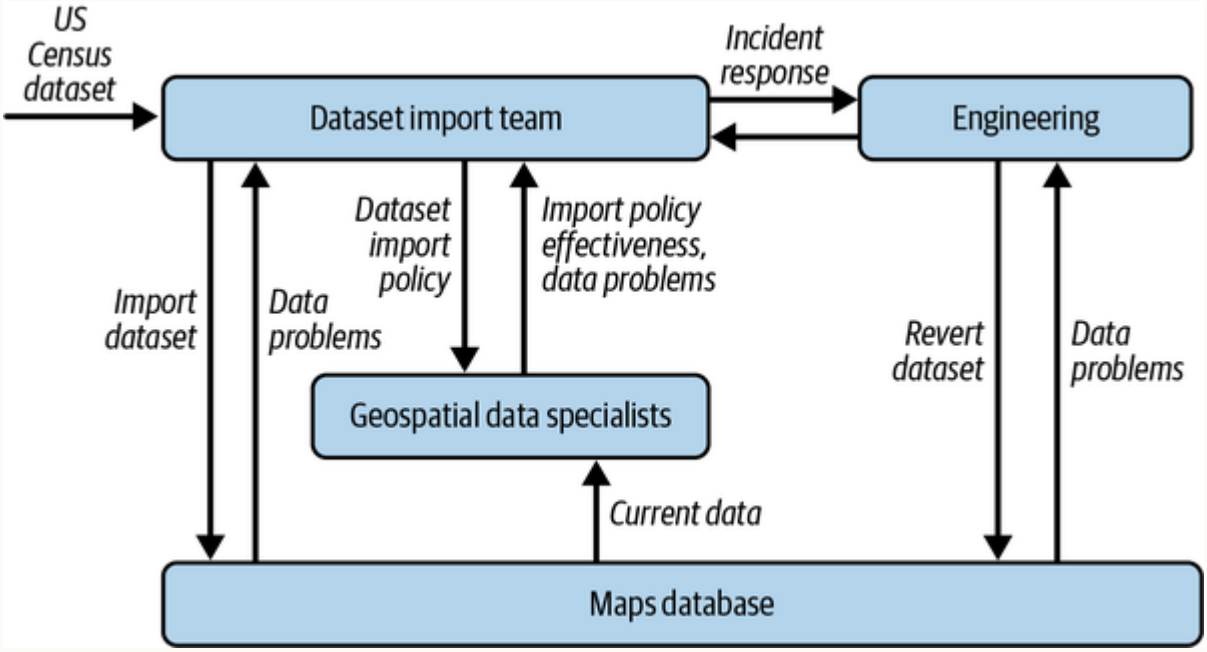


Figure 12-8. The control structure at the time of the misnamed cities incident

Our STPA example showed one part of the associated control structure, a single control–feedback loop. Here you can see that a full control structure consists of several control–feedback loops and often multiple levels of hierarchy. In a control structure, authority increases as you move up, so the dataset import team and engineering controllers have the most authority.

In CAST, it's important to specify the responsibilities of each controller that are relevant to the incident. Here's what this looks like for the misnamed cities incident:

Maps database

The Google Maps model of the real world serves as the controlled process. Our analysis primarily investigated why this model was updated with incorrect city names, leading to a hazardous state. For this purpose, the Maps database is considered to contain all relevant city data, irrespective of its underlying storage mechanisms.

Geospatial data specialists

The team of geospatial data specialists is responsible for identifying and reporting problems in candidate datasets before import into the Maps database, using its evaluation policies and tools.

Dataset import team

The dataset import team is responsible for setting the import policy. It owns the tools and authorizes dataset imports based on evaluations by the geospatial data specialists.

Engineering

The engineers are responsible for managing the data ingestion infrastructure and reverting datasets. They advised the dataset import team on data correction while the incident was ongoing.

Step 3: Analyze contributions of each controller

We assume it made sense for each controller, whether human or software, to behave the way they or it did. This avoids hindsight bias and blame, which impedes learning. As Nancy Leveson famously said, "Blame is the enemy of safety."[4] Blaming human controllers, directing them to avoid mistakes, or replacing them with someone else are not effective ways to prevent a future incident; the same system that made the original decision seem reasonable will likely lead someone else to make the same choice.

Understanding the causal factors behind unsafe behavior allows us to create more effective recommendations. Rather than simply telling people to "stick to the instructions," we can redesign the control structure—and therefore the system—to help them make safer decisions.

The first part of the individual controller analysis establishes what happened by examining the controller's responsibilities and how the controller's actions or inactions contributed to the incident. The contributions are described neutrally, avoiding subjective or judgmental words like "should have" or "failed to," which introduce hindsight bias.

The second part of the analysis explains why the behavior occurred, focusing on the mental model gaps and other contextual causal factors. The gaps in mental models, such as incorrect beliefs about the system, lead to unsafe decisions. These models are often shaped by contextual factors such as delayed feedback, misaligned incentives, ineffective training, limited resources, or conflicting goals.

When the controller is software, explaining its behavior requires examining the software's design and the designers' assumptions about how that design would ensure safe behavior, which in hindsight were incorrect.

The CAST analysis of the misnamed cities incident provided a more complete explanation of the causal factors than the postmortem. The following sidebar shows some of the

findings from the individual controller analysis of the dataset import team.

DATASET IMPORT TEAM'S CONTRIBUTIONS TO THE INCIDENT AND THEIR CAUSAL FACTORS

What happened

Responsibilities:

- Manage the entire dataset import process, including prioritization, policy setting, and authorization.
- Supervise the geospatial data specialists to ensure data and tooling issues are identified and addressed.
- Oversee post-import reviews and handle escalations.

Contributions to the incident:

- Established a data import policy primarily focused on detecting large-scale problems in the US Census dataset
- Imported the US Census dataset that contained the misnamed cities

Why? (Causal factors)

Mental model gaps:

- Believed the import policy the geospatial data specialists followed to evaluate the US Census dataset was effective
- Believed the dataset was ready to be imported

Contextual factors:

- The US Census Bureau is a trusted data source, a belief strengthened by the team's successful import of its past datasets.
- The team's assumption that a small sample would be representative, based on a history of uniformly distributed problems, led to a sampling plan that missed all six states with misnamed cities. This resulted in the false conclusion that the entire dataset was problem-free.
- The import policy's evaluation tool detected only large-scale data problems, missing combined small-scale, high-impact issues.

Notice the findings on the dataset import team's beliefs and assumptions at the time of the incident. This information, which wasn't present in the postmortem, is crucial for learning and for proposing recommendations to truly address the systemic issues. Other causal factors not identified in the postmortem included:

- The import policy lacked centralized documentation for optional tools, making it difficult for geospatial data specialists to know when to use them. They were unaware of a tool that could have detected the special character issue, since its use wasn't part of the policy.
- Lack of tooling search support for cities with "(balanced)" in their names required creating ad hoc scripts during the incident response.

CAST is a teachable and repeatable method—many people have successfully used it to discover causal factors that are often missed in a traditional postmortem. While it's possible that these causal factors could have been found in a traditional postmortem, the discovery relies heavily on the learned experience of the postmortem authors.

Step 4: Identify control structure flaws

The next step in CAST is to analyze the control structure as a whole. The goal in this step is to identify the *systemic factors* that influenced the behavior of *all* controllers, ultimately causing the system to enter the hazard state and resulting in a loss. This leads to stronger recommendations that go beyond individual unsafe controller behaviors. This analysis step can be guided by common categories of systemic factors.[5] Table 12-2 lists some of these factors and their descriptions.

Table 12-2. Some common categories of systemic factors

Systemic factor	Description
Safety culture	Examine underlying values and cultural assumptions from top management that influenced decisions contributing to the incident.
Communication and coordination	Analyze how the coordination and communication links among system components led to inconsistent, inadequate, or missing actions.
Management of change	Investigate why existing policies for planned changes were ineffective in preventing the incident.
Safety information system	Determine if controllers had complete and readily available safety-critical information required for their responsibilities.
Changes in the environment	Evaluate uncontrolled environmental changes that contributed to the incident.

Figure 12-9 illustrates a layered incident model, highlighting the critical influence of systemic factors in incident occurrence. At its core, an incident begins with adverse events, which are encircled by the behaviors of all controllers—both human and software—that produced these events. Moving outward, the next layer encompasses the causal factors that shaped the behaviors of these controllers. Finally, the outermost layer reveals the systemic factors, which create the conditions for these causal factors to exist.

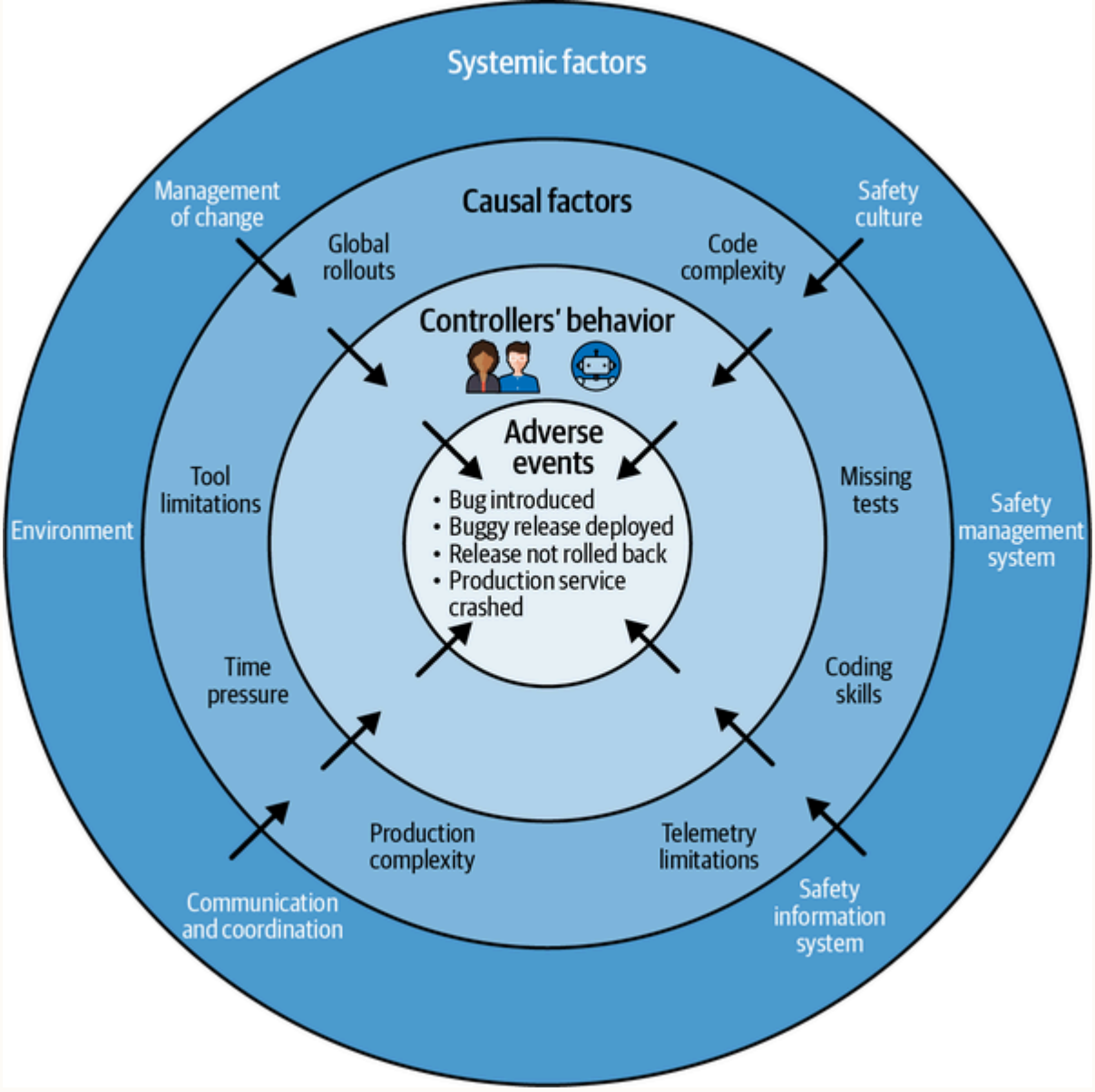


Figure 12-9. A layered model of incidents (adapted from Leveson, Engineering a Safer World, 30)

So far the CAST analysis has followed this layered model, starting with the incident events and then individual controller behaviors. However, analyzing each controller individually is insufficient. While that might prevent unsafe behavior in a single part of the system from causing an incident, systemic factors can adversely affect the behavior of all controllers at once.

By identifying these systemic factors, we gain a wider view of causality and can implement more effective measures that prevent a range of future incidents, not just the one being analyzed. For example, fixing a single causal factor—like reducing the technical debt in the part of the codebase where a bug was introduced—won't address the systemic factor of prioritizing development speed over product stability. Systemic factors like this one can manifest in other unsafe behaviors across the organization, such as superficial design reviews or neglected testing.

CAST's focus on systemic factors is the reason it is particularly well suited to finding emergent flaws due to complex component interactions. Because postmortems often

trace a chain of events, they rarely identify the top-level systemic factors that CAST is designed to uncover.

The CAST analysis of the misnamed cities identified several systemic factors, with environmental changes being the most critical.

Incidents often happen after a change. In this case, the primary systemic factor was an unplanned, gradual increase in the size of the external datasets the team received over several years. As the datasets grew, the control structure that had previously handled them effectively migrated to a higher-risk state, gradually rendering all the existing controls less effective and causing ripple effects across the entire system. Figure 12-10 illustrates this effect.

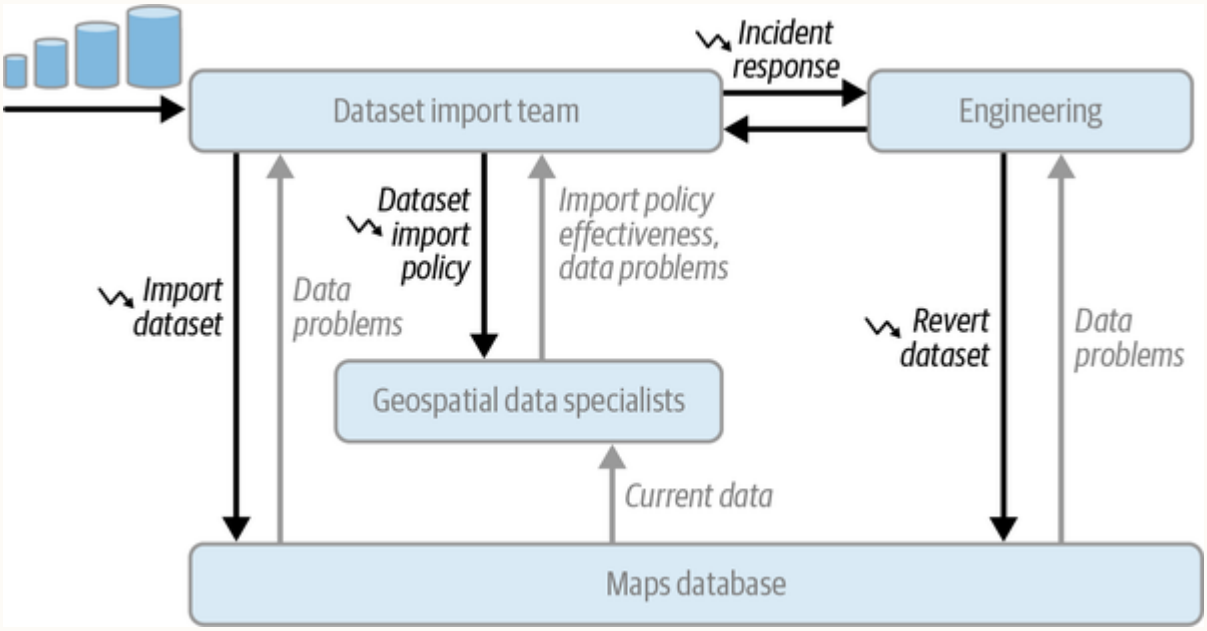


Figure 12-10. How a gradual environmental change (the increase in dataset sizes) rendered the control structure less effective and allowed unsafe behavior to occur

Here are two examples of how the existing controls became ineffective:

- Growing datasets outpaced the geospatial data specialists' import policies, which lacked clear sampling rules. The large US Census dataset could not be fully evaluated, leading to a departure from established procedures and a reliance on error-prone manual inspections.
- Outdated requirements for the geospatial data specialists' evaluation tools meant they could not effectively handle growing datasets during import policy execution. Splitting the US Census data into state-level datasets resulted in a tool designed for large-scale regressions incorrectly classifying each as "small." A "large" classification would have triggered a side-by-side evaluation, which could have detected the city name issues.

Over time, the assumption that the system design could handle datasets of any size was proven invalid. To prevent control degradation, design assumptions must be continuously monitored and revisited as the system and its environment change.

Step 5: Create an improvement plan

With a comprehensive understanding of the incident's causal and systemic factors, the objective is to formulate recommendations for safe behavior at both the component and control structure levels. This approach provides several key benefits:

- All identified factors are addressed by the recommendations.
- Clear traceability is established from the implemented recommendations back to the causal and systemic factors.
- Recommendations can be prioritized based on the urgency of the factors they address or other criteria, such as cost.

To prevent a class of incidents caused by the increasing external dataset size, CAST recommendations focused on the entire dataset import system (the control structure) rather than individual behaviors. Therefore, the CAST recommendations were more numerous and impactful than the postmortem recommendations, and each could be traced back to the causal and systemic factors it addressed. The recommendations included:

- Redesign data import policy to manage increasing volumes using structured sampling based on churn/change rates, rather than random sampling. This requires establishing methods to determine these rates.
- Create individual probe alerts for frequently rendered map entities (not just cities) to provide rapid notifications of changes in otherwise stable data.
- Implement a policy to notify data providers (e.g., US Census Bureau) of incorrect data and recommend fixes. This, despite the initial cost, improves data quality and coverage for recurring imports like the US Census dataset, benefiting all users of these public datasets.

Why Systems Theory and Safety Engineering Matters for SRE

As every SRE learns, we are only as good as our last major incident. A corollary to this is that it is difficult to talk about the value of ensuring that nothing breaks. Successful SRE teams avoid purely reactive engagements. We don't wait for things to happen before we do our jobs. At the same time, we have measures of reliability—for example, cumulative

bad customer hours, or a count of the unique users that received an error during some time period—that provide a high-level view of system reliability from the user's perspective. When we meet our goals for these measures of reliability, one of the hardest questions to answer is, "How do you know that you weren't just lucky?" How can we connect the engineering work that SRE does back to the outcomes that the business cares about? As SREs, we are always happy to benefit from luck. But we also know that "hope is not a strategy." We spend significant effort on careful and impactful engineering, designing our systems to prevent problems from ever reaching the user, and—where possible—to prevent problems from happening in the first place.

The practical challenge of safety engineering—or, if you like, "proactive reliability engineering"—in production systems is their massive complexity. By the time SREs are involved, the system is likely to have anywhere from tens to thousands of engineers working on all parts of it. The design is constantly being updated. The operational environment is changing and evolving. Products are launching, customers are using them in unexpected ways, and engineers are redeploying all of it as fast as possible.

Why STPA?

Imagine if you could travel to the future to explore the different kinds of incidents a system might experience, and then travel back to the present to prevent those failures. With a well-defined set of possible failure scenarios, you could redesign the system to eliminate most of them. The ones that remained would likely be unavoidable, due to conditions outside your control, but you would be able to design effective mitigations to limit the harm they might cause, along with feedback channels that would help humans and automation recognize exactly what mitigations were needed in context. You could ensure that if the system did start to fail in some way, all the subsequent actions would be designed to minimize the fallout from that failure. You would know exactly what to monitor, which alerts should fire and at what thresholds, and which tests to run to verify assumptions about how the system should behave.

All that may sound like impossible science fiction. It is, however, close to what you get from a thorough hazard analysis with STPA. While no method is perfect, we've seen that STPA can identify significantly more system flaws than other methods—especially those involving complex system interactions.

Hazard analysis fills gaps that exist in the other techniques that SREs use to engineer reliability into the system. It is a useful complement to these techniques, not a replacement for them. In fact, using them in combination is immensely powerful. Take Disaster Resilience Testing (see "Training and Development"), for example. A well-designed

test provides actionable information about how a system responds when failures occur. SREs can see the repercussions of a violated assumption like "the database will always be available" or "the system will fail open when the quota manager is down."

The first limitation on DiRT test effectiveness is our imagination. Coming up with testable scenarios and knowing which ones to execute is more art than science. We often realize that a test would be useful in the aftermath of a bad incident—one in which our assumptions about how the system should behave were violated in a way that maximized surprise. Of course, the old maxim that "the best time to run a DiRT test is before the incident happens, and the next best time is right now" holds true. However, using a hazard analysis technique like STPA will *uncover the assumptions in the system design* and give you all the information you need to design and execute a DiRT test without the incident actually occurring. STPA produces the details you need to execute complex sequences of actions that could lead to failures, including multistep actions, coordination between multiple controllers, feedback-driven initiation, and *n*th-order indirect effects. This rich dataset replaces the open-ended imagination process in DiRT test design with something more rigorous and comprehensive.

Another reason to add hazard analysis to the SRE discipline is that most other techniques can be applied only after a system has been built (or at least after the development process). There is no way to DiRT test a design document or to run fault injection on a design. In contrast, we can run STPA on a design that exists on paper, modeling the system as a control structure and looking for unsafe behavior just as we would in a fully implemented system.

The cost-effectiveness of catching problems during the design phase is obvious and well supported in the data. The cost of identifying and fixing a design flaw in the operational phase of a project—when the system is already running in production—is orders of magnitude higher than during the requirements or design phases. For example, a flaw that might take a team of engineers a full month to understand, fix, and deploy in a production system might take just one engineer one afternoon to fix in a design document. However, SRE has few tools that can be applied to design documents, aside from relying on experience to spot problems.

REPLICATING THE ABILITY TO FIND ISSUES IN DESIGN DOCUMENTS

A Googler relates this story:

> *As lead of a Google Docs SRE team, I worked with a senior engineer who had incredible foresight with designs. Whenever a team had a large new design, they would ask this person to be included in the review process. Without fail, the engineer would spot some significant weakness in the design. They could see when a requirement had been missed, or when the design would fail in a way that thwarted easy mitigation. This skill was valuable and much sought after, and the engineer was constantly asked to teach the skill to other SREs. They tried again and again, in design doc reading clubs and trainings. In the end, no one could come close. STPA, on the other hand, is something that anyone can learn. It's a repeatable method that can be taught and replicated across an entire organization. At Google, we've found that after we teach engineers how to do STPA, they use systems thinking to spot design issues even when they aren't doing a formal analysis.*

Another aspect of most SRE techniques that's so obvious it's easy to miss is that traditional techniques *all start with a failure*. Wheel of Misfortune trainings, fault injection, DiRT tests—these all start with something that fails or is in an abnormal state. This is so obvious that it hardly seems worth pointing out. However, if we really want to anticipate all the ways a system can fail, we have to include scenarios where no component fails but the system still can't meet its goals. This is exactly how the Google Maps incident played out. No one component failure led to the problem—the people making the update followed established procedure, and the software ran as normal, but the incident occurred anyway. A good top-down hazard analysis technique like STPA can identify cases where components work as designed but interact in a way that results in a loss.

Finally, SREs need techniques to analyze human actions. Humans are some of the most complex components in a system. As designers, we often try to push them out to the margins. It's common to see an action item in a postmortem to replace a human's manual action with automation. However, we have not fully removed humans from the loop in any of our system designs—at least, not yet! In mature production systems where we've gone through many years of automating manual interventions, the remaining human actions are all either highly abstracted (e.g., an engineer makes a tiny change to a high-level configuration, which gets transformed and expanded into a complex sequence of actions taken across the system) or exceptional (e.g., the system is in an abnormal state, and we involve humans to restore system function). In both cases, we put the humans under immense pressure:

- In the normal case (changing the high-level configuration), we ask people to reason about complex *n*th-order effects stemming from seemingly trivial changes. The system design has everything to do with how successful their actions will be.
- In the exceptional case (humans must fix an abnormal system state), we ask them to think fast about how to restore the system. Again, the system design will have everything to do with how successful they are.

We know observability is a key part of good system design. What remains difficult is determining exactly what a human needs to know to operate the system safely. With STPA, we can methodically explore what people need to know about the state of the system in order to make safe decisions.

WHAT "OBVIOUS" QUESTIONS DON'T GET ASKED?

We used STPA to analyze a design involving a significant change to Google's Remote Procedure Call (RPC) system. The change had not yet been implemented. One of the goals of the new design was to allow interoperability between all language runtimes, and between separate internal production environments. Early in the analysis, we came up with a key question: "What should happen if the system is in a state where it can't maintain interoperability?" Perhaps a change to one of the runtime libraries or a networking change leads to some part of the interoperable mesh getting cut off. The design didn't address this state. Continuing the STPA process, we raised the question of what observability channels and hedging actions needed to exist to restore full interoperability. While it might seem obvious now, as you're reading about it, this question had not previously been asked in the design process.

STPA makes problems so obvious that they often seem like they would have been trivial to find without it. Yet we know from hard-earned production experience that we deploy systems every day where the obvious questions haven't been asked.

Why CAST?

While retrospective CAST is proven in traditional safety-critical industries for maximizing learning after major incidents, applying it to our extremely complex software systems was initially challenging. At the time of our internal evaluation, there was little external literature regarding its application to software-intensive systems. This left us without tailored guidance, despite the successful applications in other domains.

However, after gaining experience using CAST to analyze several incidents, we've concluded the method is well suited to uncovering the causal factors underlying unexpected behavior in complex software systems like ours. The findings of CAST are consistently more complete than those of traditional postmortems due to two fundamental differences.

First, traditional postmortems based on linear causality result in a superficial analysis focused on event chains and root causes. CAST, which is based on the underlying STAMP causality model, moves beyond this to examine the causal and systemic factors that shaped the decisions and led to those events.

Second, because of the focus on systemic factors, CAST-driven improvement plans directly reflect the systemic and organizational nature of the findings. It's common to see recommendations that address decision making at the top of the control structure—the high-level design and policy decisions made by management and leadership, which provide the greatest leverage for controlling the system behavior. Examples of these may include:

- Completely overhauling an internal process. For example, the misnamed cities incident required a full redesign of the existing dataset evaluation process. Other examples include overhauling the end-to-end development lifecycle, improving how SRE and product development teams work together, or revising the hiring rubric.
- Restaffing and clarification of team responsibilities. For example, CAST sometimes reveals that responsibilities were misaligned or inadequately fulfilled following a previous internal restructuring.
- Measuring the impact and completeness of post-incident improvement plans across the company. For example, you might want to measure whether improvements are actually implemented and whether they are effective.

In contrast, traditional postmortems based on linear causality often produce recommendations focused narrowly on treating the symptoms of individual events. While these localized improvements are necessary, they are insufficient on their own because they fail to address the underlying systemic factors. Examples of such postmortem recommendations typically include:

- Adding missing tests to prevent bug reintroduction—but what about the development practices that allowed the bug in the first place?
- Transferring on-call support to a new team to ensure service availability response—but what about the company-level support model that allows understaffed teams to inherit business-critical services?

- Fixing the performance regression that increased API latency—but what about the production readiness evaluation that fails to enforce performance targets for a public API launch?

Incident postmortems allow you to inspect the spills and shattered glass on the table's surface. The focus is on cleaning up the mess (fixing the events) and applying a fresh protective coat to that specific area, hoping the same mess won't happen again.

CAST not only notes the spills and shattered glass of the surface, but it also forces you to look under the table and examine its structural foundation. CAST asks: "Why is the table wobbly? Why were the supporting legs (the control structure) misaligned? Why did we design a table where spills were inevitable?"

In short: postmortems often only fix the symptoms of a wobbly table; CAST ensures a better design for the next table.

Contrasting Traditional SRE Methods with STPA/CAST

To conclude this section, Table 12-3 summarizes, based on our experience, how safety engineering, STPA, and CAST differ from traditional SRE methods.

Table 12-3. Traditional SRE methods versus safety engineering (STPA/CAST)

Traditional SRE methods	Safety engineering (STPA/CAST)
Incidents are inevitable—invest in monitoring, tooling, and training to quickly detect and mitigate incidents when they happen.	Anticipate incidents by using STPA to identify system flaws, then prioritize and fix the worst flaws before they can cause incidents.
Do postmortems and follow up on the findings—often, we are "treating the symptoms" (reactive).	Look at the system as a whole, and identify and fix the causal factors that could lead to an incident before it happens (proactive).
Design systems according to tried and true best practices and production principles, which we know from experience reduce incidents.	Perform hazard analysis on designs and identify missing requirements/gaps that must be addressed to ensure safe system behavior.
Identify problems from the bottom up, using discrete events to identify the general class of problem across a system.	Identify problems from the top down, starting with what must be true for the system to operate correctly and finding all aspects of system behavior where this is not the case.
Address all the root causes identified in the postmortem—hopefully, this is sufficient to prevent recurrences of the incident.	Apply CAST to identify the causal factors and systemic factors that contributed to the decisions made in the incident—addressing those factors will prevent whole classes of incidents.
A blameless postmortem culture establishes psychological safety and thus maximizes learning.	Implement an *effective* postmortem culture that is both blameless and analytical—don't judge individuals for their actions, and use CAST to identify the factors that influenced their decisions.

Driving Adoption of STPA and CAST

In this section, we present some lessons we've learned as we have introduced forward-looking STPA and retrospective CAST to Google SREs. More than just introducing techniques, this involves a fundamental shift to systems thinking that, in many cases, changes how engineers look at a production system or design—even if they only participate in a single analysis.

Introducing STPA to Your Organization

Google has seen significant benefit from applying STPA, even in a limited capacity. The Google Maps analysis after the incident in Japan is one of many success stories. Since introducing STPA at Google, we've worked to increase awareness, build training, find sponsors, and convince people that it is an effective tool for proactively anticipating software incidents, with high return on investment. We've also learned about challenges and effective strategies for STPA adoption from proponents in other industries and academia.

One of the biggest challenges with pitching STPA to your organization is that its value is not immediately clear or easy to grasp. The value of STPA is diverse and nuanced—it includes the new perspective gained from systems thinking, a new way to model software systems that enables deeper understanding, a method to thoroughly explore the space of unknown unknowns, and more. For someone involved in making decisions about resources and funding who sees many project proposals in a given planning period, a 15-minute pitch on STPA might initially make it sound like a miracle cure, leading to skepticism. We have encountered many industry stories of STPA advocates working for years to convince their organizations to adopt it—this is not an isolated challenge.

Starting from the top, one crucial selling point for adopting STPA is the immense return on investment achieved by applying it during the design phase. Leaders universally recognize that learning from incidents is inherently expensive, with costs manifesting in two ways: the direct impact cost (lost revenue, damaged customer trust, regulatory exposure) and the often more significant opportunity cost as engineering bandwidth is diverted from feature development to implementing postmortem remediation tasks. This illustrates the hidden cost of incidents: the cost of design flaws increases exponentially over time. Figure 12-11 visually confirms this principle, demonstrating qualitatively how the opportunity cost of fixes increases as flaw discovery shifts later in the development cycle.

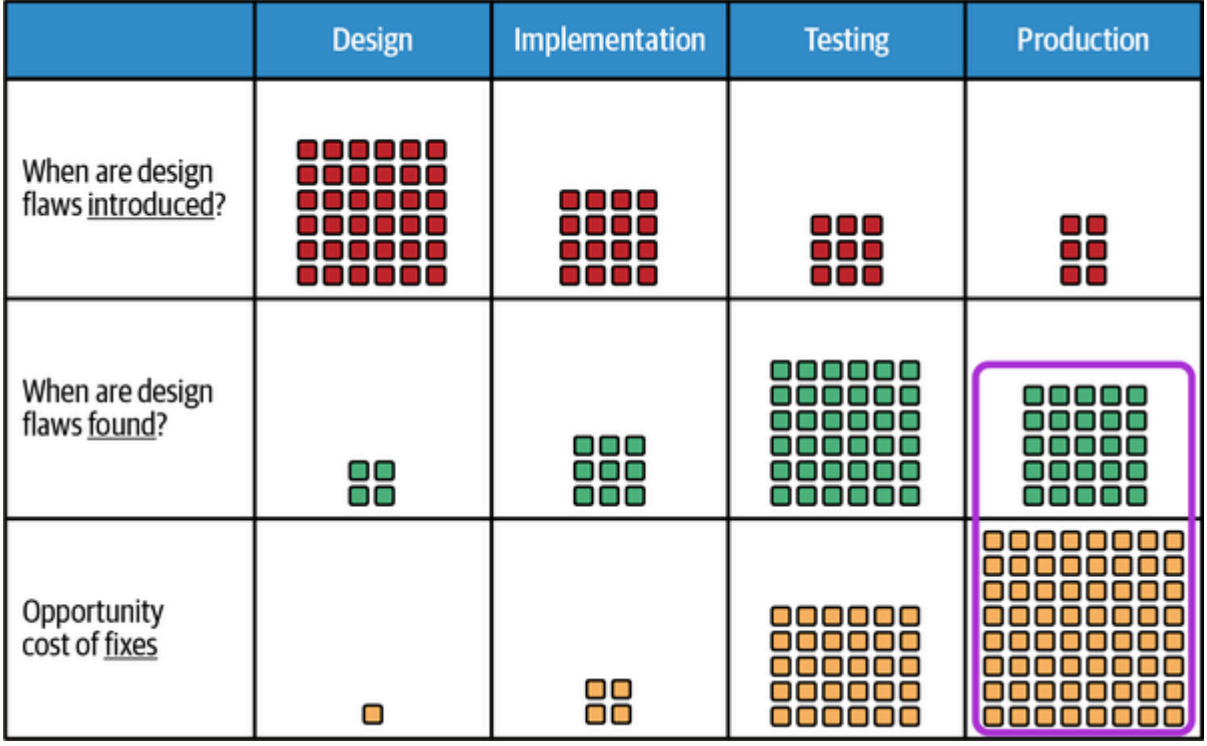


Figure 12-11. The hidden cost of incidents (coarse approximation, adapted for Google from John Thomas's "System Safety and STPA" class materials [2021] based on data from the 2018 SEI report "ROI Analysis of the System Architecture Virtual Integration Initiative")

So here's the challenge: if a postmortem teaches us where we have gaps or flaws in our system design, can we move that learning earlier in the product lifecycle so that it happens at the cheapest possible point? Or at the very least, can we move the learning up so that it occurs before an outage happens, allowing us to avoid the crisis engineering mode?

By leveraging STPA to anticipate incidents during the design phase, we minimize opportunity cost—a concept illustrated qualitatively in Figure 12-12. We can demonstrate to leaders that the marginal investment in applying STPA during design (slightly increased opportunity cost) yields a substantial ROI over time. By preemptively preventing future incident costs, we ensure engineering resources (otherwise spent on postmortem fixes) are immediately redirected to core feature development. In the end, STPA allows us to trade fewer incidents for more features.

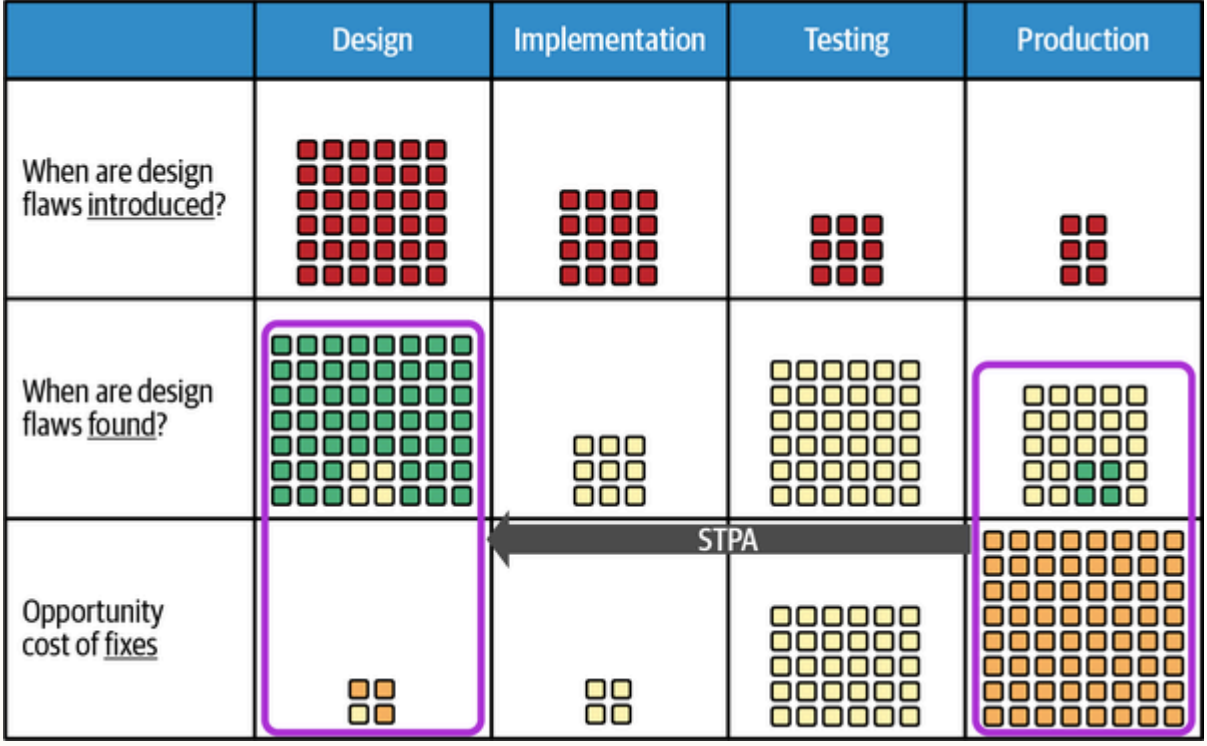


Figure 12-12. By shifting design flaw discovery to the earliest stages of the software lifecycle, STPA minimizes the opportunity cost of production fixes

In our experience, we've also achieved success from the bottom up: working with individual contributors who see the value of STPA, are willing to try it out, and later promote its effectiveness to their peers and managers. Rather than pitching STPA and trying to explain how it works, we've had more success presenting a successful analysis and a significant finding that nobody had discovered previously. After the impact of this presentation, we then add, "By the way, we found this using a new technique that's really good at anticipating incidents due to complex system interactions." As more teams hear about STPA and run successful analyses, and the demand for STPA grows, it becomes easier to find sponsors. As you start introducing STPA to your organization, identify the innovators—people who immediately see value and potential, and want to learn more or try it out on their own systems.

As you talk to others about STPA, you'll encounter people who have heard or read something about why it doesn't work or is impractical. While there is certainly a learning curve and it takes effort to learn how to apply STPA successfully to software systems, you'll also encounter some common misconceptions, and it helps to have some context when you hear these arguments. Importantly, many of these misconceptions are based on truths *in other industries*. Companies that design safety-critical systems, such as aircraft or manufacturing equipment, are often required by law to perform extensive safety engineering and fault analysis. These companies can afford to, for example, spend a year exhaustively analyzing and certifying an aircraft that will then be in service for many years. In contrast, the software industry is notorious for its fast pace of feature development and releases. The good news is that those of us in the software industry can still achieve

tremendous benefit from applying STPA, even in a reduced fashion. With this background in mind, we present several misconceptions and their associated clarifications:

Misconception: An STPA project requires weeks or months of investment from many engineers.

This misconception likely comes from STPA's original application to safety-critical systems, such as in the aerospace and nuclear power industries. In these industries, where design flaws can lead to loss of life, there are strict requirements for safety analysis, and months of exhaustive analysis are justified. Outside of these requirements, the duration of an STPA project is primarily governed by scope and SME availability. For noncritical software and human systems, you can reduce the time required to run STPA by narrowing the scope of the analysis—for example, by prioritizing only the most critical UCAs. At Google, thanks to investment in training a few STPA experts and increased efficiency through experience, we have successfully completed limited-scope STPAs in less than a week, with only one or two SMEs.

Misconception: STPA does not identify novel design flaws.

When we run STPA, domain experts outside the analysis often look at the results and say that the identified design flaws are obvious. This is another example of hindsight bias. Our experience shows that when SMEs participate in STPA and model their systems using control–feedback loops, they frequently find design problems that were previously unknown. This method is effective for two main reasons. First, it shifts the focus from data flow (what the system does to achieve its goals) to inadequate control (what the system is not doing to prevent incidents). This is a helpful contrast to traditional analyses that focus only on how data moves through the system. Second, STPA allows for the analysis of interactions between *people* and software, in contrast to traditional analyses that focus on just the software parts. These human factors analyses are a significant source of novel design flaw discoveries.

Misconception: If I run STPA, I'll end up with a huge list of issues and vulnerabilities. My backlog is filled with fixes for things I already know about, so it will be difficult or impossible to justify addressing all those issues.

Again, you don't necessarily have to run an exhaustive STPA. You might end up with only a few findings, but those findings might involve significant risks! However, there's a more subtle argument here: suppose you do end up with a long list of findings. While seeing so many vulnerabilities listed together might be scary, it's also really powerful—you might realize that the known risk you've been

spending significant resources to mitigate pales in comparison to a newly discovered risk. Even more impactful, we often discover patterns in STPA results where a single mitigation eliminates an entire class of vulnerabilities.

Additionally, applying STPA during the design phase can produce a list of design flaws that are often simple to resolve, sometimes requiring only minor edits to the design document or diagrams. Making these simple fixes at this stage represents the most economical use of engineering resources, compared to more costly incident response and post-incident remediation efforts.

Finally, consider the core objective of your analysis: is the goal to maximize learning about the system, or merely to minimize the number of required mitigations? If the objective is learning, then a comprehensive list of causal factors and associated required mitigations is expected. Moreover, not all required mitigations need immediate action; some will be quick wins, while others will be long-term projects. Therefore, the outputs of STPA are useful for organizing your plans and allocating resources.

Misconception: To lead an analysis using STPA, you need years of formal training.

While it's true that you can't learn to run a maximally efficient and effective STPA process from a single tutorial or class, you can still achieve benefits from your first efforts to apply STPA. The Maps data updates approval STPA, described earlier in this chapter, is an example of an early STPA we ran. The Googlers running the analysis had little experience with STPA, and although it took longer than it might have with an experienced team, the analysis was still successful. When you're in the early stages of learning STPA, it's important to seek expert guidance (e.g., the courses and certification from the STAMP Institute) to ensure that you are following the methodology correctly. In our case, this initial success provided the momentum to keep going with additional STPA projects in other areas. Since these initial analyses, we have gained enough experience to run STPA on our own, and we are continually improving our efficiency and effectiveness.

You've seen some of the common friction points and arguments against STPA, and you're ready with counterarguments. Now you're ready to try it for yourself to hopefully get some results you can use to convince others. There are two important points to keep in mind: choose the system to analyze wisely, and fail fast.

Choosing a system and scope for your first STPA analysis can mean the difference between impactful results and frustration. Here are a few suggestions based on our experience:

- Don't choose a system that's so simple and/or so well understood that the outcome of STPA (or any risk analysis, for that matter) will be obvious.
- Instead, choose a system that has previously demonstrated unexpected behavior or unanticipated complex interactions, and is likely to do so again—STPA excels at analyzing this kind of complexity.
- To maximize impact, identify system areas of concern. Choose areas with high-consequence problems, like risky phases of operation, or systems where you suspect there are many "unknown unknowns."
- Choose a system that includes human interaction with automated controls—this often leads to interesting outcomes.
- Optionally, choose a system in the conception or early design phase. STPA is a powerful tool for identifying safety risks and design issues before a system has been built, and it makes a big impression when you can eliminate incidents just by changing a few sentences in a design document!

When getting started with STPA, also keep the principle of "fail fast" in mind. Start small, with a limited analysis and two or three participants. If the first analysis doesn't produce interesting results, you can quickly run another one. Once you have gained experience and are consistently discovering new risks and system vulnerabilities, you can start running larger analyses. It's better to fail fast after a day or two than to spend a few weeks running your first STPA with 10 or more engineers, only to have little to show for it—that kind of experience can significantly set back STPA adoption.

Adopting CAST: Overcoming Tradition

Having established the value of CAST over traditional postmortems, the next step is introducing it to an organization. This can be challenging in companies that have a long tradition of postmortems and because of the organizational costs of adopting a new analysis paradigm.

One way to encourage the early adoption of CAST is by demonstrating the method's value in three key areas: impact, scalability, and efficiency.

Impact

There are two key ways to demonstrate that CAST produces more significant improvements than a traditional postmortem based on linear causality:

Tangible engineering projects

CAST analyses identify actionable engineering projects to reduce system risk. Leadership buy-in and resources are essential to implement recommendations, as insights are useless without action.

Side-by-side comparisons

Comparing CAST and postmortem results highlights CAST's ability to reveal deeper insights and generate more impactful recommendations.

Scalability

Leaders often question whether CAST is scalable enough for complex sociotechnical systems. We've demonstrated its scalability by successfully applying it to incidents involving hundreds of software and human components. We maintain a portfolio of these successful analyses as supporting evidence.

Efficiency

Leaders also want to know if CAST can be executed quickly in a fast-paced environment. Although CAST can take longer than a traditional postmortem, we mitigate this by sharing the findings and recommendations as they are produced. This provides rapid insights without waiting for a final report. We also provide descriptive statistics on the effort required for CAST and postmortems to illustrate the return on investment of both methods over time.

The CAST analysis of the misnamed cities incident described earlier showed that CAST is more expensive in terms of time, but it provides many more insights than a traditional postmortem (see Table 12-4). In addition, the insights from CAST are broader in scope than what we find when we look at the software components alone. CAST identified much deeper systemic issues that contributed to the incident, enabling us to see system-wide flaws and longer-term issues.

Table 12-4. Resource cost-benefit comparison of CAST and postmortems

	Postmortem	CAST
Number of people	3	5
Total hours	16	81
Number of findings	6	7
Number of recommendations	8	22

Conclusion

This chapter provided an overview of how forward-looking STPA and retrospective CAST work, and why SREs should be interested. We showed the impact these methods can have through real-world examples, and we discussed some of the challenges to adopting STPA and CAST in an organization along with ways to address these challenges. We can't teach you how to run STPA or CAST in a single chapter, but there are many online resources that can help you build knowledge and even achieve certification. For more information, see the MIT Partnership for Systems Approaches to Safety and Security.

STPA and CAST have the potential to transform Site Reliability Engineering. As Ben Treynor Sloss discussed in season 3, episode 3 of *The Prodcast*, STPA shifts proactive reliability toward pure engineering:

> *The idea of, "I can tell you all of the outages that you're going to have before they happen, and then you can decide how many of them you want to fix and how many of them you want to live with" as sort of an engineering choice, that's incredibly interesting!*

We encourage you to learn more about STPA and CAST and try them out—even starting small, you can make an impact through greater understanding of your complex systems and gaining the opportunity to fix system issues before they cause incidents.

1 Sidney Dekker, *The Field Guide to Understanding "Human Error"*, 3rd ed. (Ashgate, 2014).

2 Nancy G. Leveson, *CAST Handbook: How to Learn More from Incidents and Accidents*, 2019.

3 Betsy Beyer et al., *Site Reliability Engineering: How Google Runs Production Systems*, 1st ed. (O'Reilly, 2016), 169.

4 Nancy G. Leveson, *Engineering a Safer World: Systems Thinking Applied to Safety* (MIT Press, 2011), 18.

5 Nancy G. Leveson, *An Introduction to System Safety Engineering* (MIT Press, 2023), 79.

‹ 11. Learning from Incidents

IV. Infrastructure Design ›